UNIT-III

SQL

SQL **ORDER BY** clause

SQL **ORDER BY** clause

1.The SQL ORDER BY clause is used for **sorting data in ascending and descending** order based on one or more columns.

2.Some databases sort query results in **ascending order by default**.

SQL ORDER BY syntax:

```
SELECT expressions
FROM tables
WHERE conditions
ORDER BY expression [ASC | DESC];
```

CUSTOMERS table

Let us take a CUSTOMERS table having the following records:

ID	NAME	AGE	ADDRESS	SALARY
1	Himani gupta	21	Modinagar	22000
2	Shiva tiwari	22	Bhopal	21000
3	Ajeet bhargav	45	Meerut	65000
4	Ritesh yadav	36	Azamgarh	26000
5	Balwant singh	45	Varanasi	36000
6	Mahesh sharma	26	Mathura	22000

Example

This is an example that would sort the result in ascending order by NAME and SALARY.

```
SELECT * FROM CUSTOMERS
ORDER BY NAME;
```

ID	NAME	AGE	ADDRESS	SALARY
3	Ajeet bhargav	45	Meerut	65000
5	Balwant singh	45	Varanasi	36000
1	Himani gupta	21	Modinagar	22000
6	Mahesh sharma	26	Mathura	22000
4	Ritesh yadav	36	Azamgarh	26000
2	Shiva tiwari	22	Bhopal	21000

Example

This is an example to sort the result in descending order by NAME.

```
SELECT * FROM CUSTOMERS
ORDER BY NAME DESC;
```

ID	NAME	AGE	ADDRESS	SALARY
2	Shiva tiwari	22	Bhopal	21000
4	Ritesh yadav	36	Azamgarh	26000
6	Mahesh sharma	26	Mathura	22000
1	Himani gupta	21	Modinagar	22000
5	Balwant singh	45	Varanasi	36000
3	Ajeet bhargav	45	Meerut	65000

SQL Aggregate Functions

Aggregate functions

1.Aggregate functions perform a calculation on a set of values and return a single value.
2.Aggregate functions ignore NULL values except COUNT.
3.It is used with the GROUP BY clause of the SELECT statement.

- **AVG()** - The SQL AVG aggregate function selects the average value for certain table column.
- **COUNT()** - The SQL COUNT aggregate function is used to count the number of rows in a database table.
- **MAX()** - The SQL MAX aggregate function allows us to select the highest (maximum) value for a certain column.
- **MIN()** - The SQL MIN aggregate function allows us to select the lowest (minimum) value for a certain column.
- **SUM()** - The SQL SUM aggregate function allows selecting the total for a numeric column.

Example

<Employee> Table

Eid	Ename	Age	City	Salary
E001	ABC	29	Pune	20000
E002	PQR	30	Pune	30000
E003	LMN	25	Mumbai	5000
E004	XYZ	24	Mumbai	4000
E005	STU	32	Bangalore	25000

Example

Aggregate Functions	Description	Syntax	Example	Output
AVG	It returns the average of the data values.	SELECT AVG <column_name> FROM <table_name>;	SELECT AVG(Salary) FROM Employee;	**AVG(Salary)** 16800

Example

Aggregate Functions	Description	Syntax	Example	Output
MAX	It returns the maximum value for a column.	SELECT MAX <column_name> FROM <table_name>;	SELECT MAX(Salary) FROM Employee;	**MAX(Salary)** 30000

Example

Aggregate Functions	Description	Syntax	Example	Output
MIN	It returns the minimum value for a column.	SELECT MIN <column_name> FROM <table_name>;	SELECT MIN(Salary) FROM Employee;	**MIN(Salary)** 4000

Example

Aggregate Functions	Description	Syntax	Example	Output
SUM	It returns the sum (addition) of the data values.	SELECT SUM <column_name> FROM <table_name>;	SELECT SUM(Salary) FROM Employee WHERE City='Pune';	**SUM(Salary)** 50000
COUNT()	It returns total number of values in a given column.	SELECT COUNT <column_name> FROM <table_name>;	SELECT COUNT(Empid) FROM Employee;	**COUNT(Empid)** 5
COUNT(*)	It returns the number of rows in a table.	SELECT COUNT(*) FROM <table_name>;	SELECT COUNT(*) FROM Employee;	**COUNT(*)** 5

Example::employee_tbl

Consider an **employee_tbl** table, which is having the following records:

```
SQL> SELECT * FROM employee_tbl;
+------+------+------------+--------------------+
| id   | name | work_date  | daily_typing_pages |
+------+------+------------+--------------------+
|    1 | John | 2007-01-24 |                250 |
|    2 | Ram  | 2007-05-27 |                220 |
|    3 | Jack | 2007-05-06 |                170 |
|    3 | Jack | 2007-04-06 |                100 |
|    4 | Jill | 2007-04-06 |                220 |
|    5 | Zara | 2007-06-06 |                300 |
|    5 | Zara | 2007-02-06 |                350 |
+------+------+------------+--------------------+
7 rows in set (0.00 sec)
```

SQL AVG function

AVG() Syntax

```
SELECT AVG(column_name)
FROM table_name
```

```
SQL> SELECT AVG(daily_typing_pages)
    -> FROM employee_tbl;
```

```
AVG(daily_typing_pages)
230.0000
```

SQL **GROUP BY** clause

1.The SQL **GROUP BY** clause is used in **collaboration with the SELECT** statement to **arrange identical data into groups.**

2.The GROUP BY statement is often **used with aggregate functions (COUNT, MAX, MIN, SUM, AVG)** to group the result-set by one or more columns.

3.The GROUP BY clause follows the WHERE clause in a SELECT statement and precedes the ORDER BY clause.

SQL GROUP BY clause

The SQL GROUP BY syntax

The general syntax is:

```
SELECT column-names
FROM table-name
WHERE [condition]
GROUP BY column-names
```

The general syntax with ORDER BY is:

```
SELECT column-names
FROM table-name
WHERE condition
GROUP BY column-names
ORDER BY column-names
```

Example:

Consider the CUSTOMERS table

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

Example:

If you want to know the total amount of the salary on each customer, then the GROUP BY query would be as follows.

```
SQL> SELECT NAME, SUM(SALARY)
        FROM CUSTOMERS
        GROUP BY NAME;
```

NAME	SUM(SALARY)
Chaitali	6500.00
Hardik	8500.00
kaushik	2000.00
Khilan	1500.00
Komal	4500.00
Muffy	10000.00
Ramesh	2000.00

Example:

For Example: If you want to know the total amount of salary spent on each department, the query would be:

```
SELECT dept, SUM (salary)
FROM employee
GROUP BY dept;
```

For Example:

```
SELECT location, dept, SUM (salary)
FROM employee
GROUP BY location, dept;
```

SQL HAVING Clause

1.The HAVING clause is generally used along with the GROUP BY clause.

2.HAVING clause is used to filter the results obtained by the GROUP BY clause based on some specific conditions.

3.HAVING clause is quite similar to the WHERE clause as both are used to filter records in SQL queries.

4.But WHERE clause can not be used with aggregate functions (Eg: COUNT, MAX, SUM, etc) which is why HAVING clause is needed.

For-example:

```
SELECT dept, SUM (salary)
FROM employee
GROUP BY dept
HAVING SUM (salary) > 25000
```

SQL HAVING Clause

Example 1:

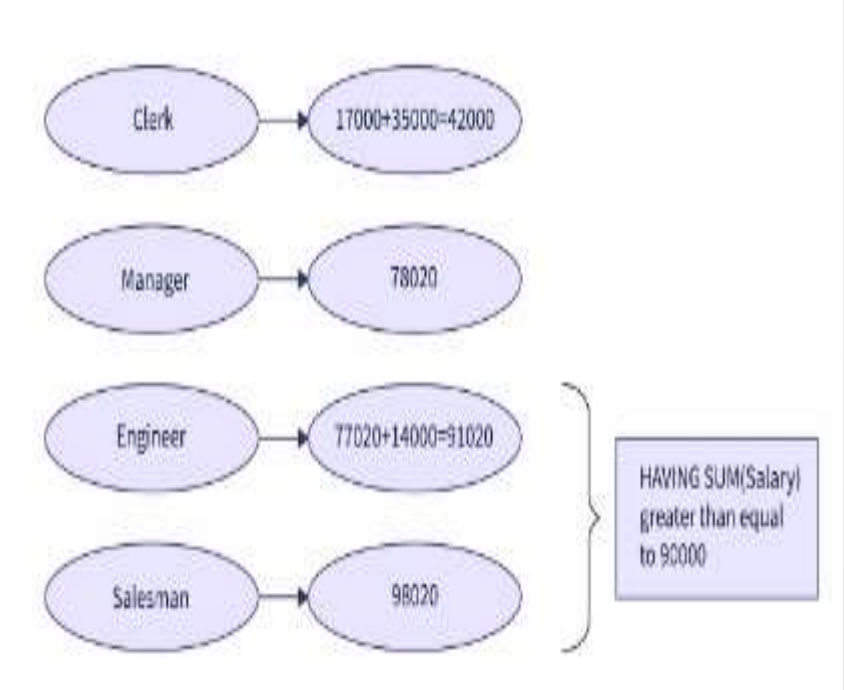
Consider the following table named "Employees".

EmpNo	EName	Job	Salary	DeptNo	Age
1	John	Clerk	17000	10	25
2	Harry	Clerk	35000	20	27
3	David	Manager	78020	50	26
4	Smith	Engineer	77020	10	35
5	Clarke	Salesman	98020	20	32
6	Musk	Engineer	14000	50	45

SQL HAVING Clause

Query 1: Let's say we want to display the Job types with total sum of salary greater than or equal to 90000. Then the query will be:

```
Select Job, SUM(Salary)
FROM Employees
GROUP BY Job
HAVING SUM(Salary)>=90000;
```



SQL HAVING Clause

Query 1: Let's say we want to display the Job types with total sum of salary greater than or equal to 90000. Then the query will be:

The output for the above query is given below

Job	SUM(Salary)
Engineer	91020
Salesman	98020

SQL HAVING Clause

Query 2: Let's apply one more condition on the above query which is the age of the employee should be greater than or equal to 35.

```
SELECT Job, SUM(Salary)
FROM Employees
WHERE Age>=35
GROUP BY Job
HAVING SUM(Salary)>=90000;
```

The output for the above query is given below.

Job	SUM(Salary)
Engineer	91020

SQL HAVING Clause

Query 3: Let's take another example where we want to display the Job types with total sum of salary greater than or equal to 60000. Here we will **use ORDER BY clause** along with the HAVING clause to sort the output according to the salary.

```
SELECT Job, SUM(Salary)
From Employees
GROUP BY Job
HAVING SUM(Salary)>=60000
ORDER BY SUM(Salary);
```

The output for the above query is given below

Job	SUM(Salary)
Manager	78020
Engineer	91020
Salesman	98020

SQL Functions

SQL Functions

SQL functions sometimes take arguments and always return a value.

There are two types of SQL functions are used in queries

- **Single-row functions** These functions operate on single rows only and return one result per row.

- **Multiple-row functions** These functions can manipulate groups of rows to give one result per group of rows.

SQL Functions

There are different types of single-row functions as described below:

1. •**Character→** Accept character input and can return both character and number values

2. • **Number→** Accept numeric input and return numeric values

3. •**Date→** Operate on values of the DATE data type.
   All date functions return a value of the DATE data type except the MONTHS_BETWEEN function, which returns a number.

SQL Functions

DUAL Table

1. The DUAL table is owned by the **user SYS** and can be accessed by all users.
2. It contains one column, DUMMY, and one row with the **value X.**
3. The DUAL table is **useful when you want to return a value only once**

```
1  SELECT * FROM DUAL;
```

DUMMY
X

SQL Functions

Number Functions

Number functions accept numeric input and return numeric values.

Function	Meaning	Use	O/P
ABS(n)	Returns absolute value (+ve equivalent)	Abs(-10)	10
CEIL(n)	Returns smallest integer greater than or equal to n.	Ceil(10.2)	11
FLOOR(n)	Returns largest integer equal to or less than n.	Floor(10.6)	10
MOD(n2,n1)	Returns the remainder of n2 divided by n1. Returns n2 if n1 is 0.	Mod(5,2)	1
POWER(n2,n1)	Returns n2 raised to the n1 power.	Power(8,2)	64
ROUND(number)	Returns rounded number.	Round(10.7)	11
TRUNC(number)	Return truncated value of number.	Trunc(10.7)	10
SQRT(n)	Returns the square root of n.	Sqrt(64)	8

SQL Functions

Number Functions

```
1  SELECT ABS(-10) from dual;
```

ABS(-10)
10

```
1  SELECT CEIL(25.75) from dual;
```

CEIL(25.75)
26

```
1  SELECT FLOOR(25.75) from dual;
```

FLOOR(25.75)
25

```
1  SELECT MOD(18, 4) from dual;
```

MOD(18,4)
2

SQL Functions

Number Functions

```
1  SELECT TRUNC(7.53635, 2) from dual;
```

TRUNC(7.53635,2)
7.53

```
1  SELECT SQRT(25) from dual;
```

SQRT(25)
5

```
1  SELECT ROUND(5.553) from dual;
```

ROUND(5.553)
6

SQL Functions

Character Functions

Single-row character functions **accept character data as input** and can **return both character and numeric values**.

Character functions can be divided into the following:

1. • **Case-conversion functions**

2. • **Character-manipulation functions**

SQL Functions

Character Functions

Case-Conversion Functions

LOWER, UPPER, and INITCAP are the three case-conversion functions.

1. **LOWER:** Converts mixed-case or uppercase character strings to lowercase

2. **UPPER:** Converts mixed-case or lowercase character strings to uppercase.

3. **INITCAP**: Converts the first letter of each word to uppercase and the remaining letters to lowercase

1.lower ():

lower (): This function converts the uppercase letters to lower case letters what you are passed to the function.

Syntax: lower(message)

Example

select lower('KEERTHI') as low from dual;

```
SQL> select  lower('KEERTHI') as low from dual;

LOW
-------
keerthi
```

2.upper():

upper(): this function is used to convert the lower case letters into uppercase letters.

Syntax: upper(message)

Example

select upper('database') as upper1 from dual;

```
SQL> select upper('database') as upper1 from dual;

UPPER1
--------
DATABASE
```

3. initcap():

3. initcap():

It make initial letter to capital letter what you have passed to the function.

Syntax: initcap(message)

select initcap('keerthi') from dual;

```
SQL> select  initcap('keerthi') from dual;

INITCAP
-------
Keerthi
```

SQL Functions

Character Functions

Character-Manipulation Functions

1. **CONCAT:** Joins values together (You are limited to using two parameters with CONCAT.)
2. **SUBSTR:** Extracts a string of determined length
3. **LENGTH**: Shows the length of a string as a numeric value
4. **INSTR:** Finds the numeric position of a named character
5. **LPAD:** Returns an expression left-padded to the length of n characters with a character expression
6. **RPAD:** Returns an expression right-padded to the length of n characters with a character expression
7. **TRIM:** Trims leading or trailing characters (or both) from a character

SQL Functions

Character Functions

Character-Manipulation Functions

Function	Result
CONCAT('Hello', 'World')	HelloWorld
SUBSTR('HelloWorld',1,5)	Hello
LENGTH('HelloWorld')	10
INSTR('HelloWorld', 'W')	6
LPAD(salary,10,'*')	*****24000
RPAD(salary, 10, '*')	24000*****
REPLACE ('JACK and JUE','J','BL')	BLACK and BLUE
TRIM('H' FROM 'HelloWorld')	elloWorld

SQL Functions

Character Functions

```
Input1: SELECT CONCAT('computer' ,'science') FROM DUAL;
Output1: computerscience

Input2: SELECT CONCAT( NULL ,'Android') FROM DUAL;
Output2: Android

Input3: SELECT CONCAT( NULL ,NULL ) FROM DUAL;
Output3: -
```

```
1   SELECT INITCAP('computer science portal ') FROM DUAL;
```

INITCAP('COMPUTERSCIENCEPORTAL')
Computer Science Portal

SQL Functions

Character Functions

```
Input1: SELECT LENGTH('Learning Is Fun') FROM DUAL;
Output1: 15

Input2: SELECT LENGTH('   Write an Interview  Experience ') FROM
DUAL;
Output2: 34
```

```
Input1: SELECT SUBSTR('Database Management System', 9) FROM DUAL;
Output1: Management System

Input2: SELECT SUBSTR('Database Management System', 9, 7) FROM
DUAL;
Output2: Manage
```

SQL (String conversion)-String Functions

L_NUMBER	F_NAME	L_NAME	STREET	CITY	STATE
1	Arun	Ravi	DNR	BVRM	AP
2	Ajay	Kiran	JP Road	BVRM	AP
3	Satish	Sandeep	MG Road	BZA	AP
4	Sandhya	Rani	KrishnaLanka	BZA	AP
5	Geetha	Manaswi	BesantRoad	BZA	AP

Ravi

SQL functions(Date and Time, Numeric, String conversion)-String Functions

- **LENGTH():** This string function returns the length of the specified word. It shows the number of characters from the word.
- **Syntax:**
  - SELECT LENGTH(Column_Name) as Alias_Name FROM Table_Name;
- **Example:**
  - SELECT F_Name,LENGTH(F_Name)as Lenght_Of_Name FROM Student1;

SQL functions(Date and Time, Numeric, String conversion)-String Functions

	F_NAME	LENGHT_OF_NAME
1	Arun	4
2	Ajay	4
3	Satish	6
4	Sandhya	7
5	Geetha	6

SQL functions(Date and Time, Numeric, String conversion)-String Functions

- **CONCAT():** This string function concatenates two strings or words and forms a new string in the result.
- **Syntax:**
  - **SELECT** CONCAT(Column_Name1, Column_Name2, ... column_NameN) **AS** Alias_Name **FROM** Table_Name;
- **Example:**
  - SELECT CONCAT(F_Name, ' ', L_Name) AS fullName FROM Student1;

FULLNAME
Arun Ravi
Ajay Kiran
Satish Sandeep
Sandhya Rani
Geetha Manaswi

SQL functions(Date and Time, Numeric, String conversion)-String Functions

- **SUBSTRING():**
- The SUBSTRING SQL string function extracts a substring or a portion of a string from a larger string.
- It takes three arguments: the string to be removed from, the substring's starting position, and the substring's length.
- **Syntax** SELECT substr(L_Name,1,3) AS SUBStringExample FROM Student1;

SUBSTRING
Rav
Kir
San
Ran
Man

SQL functions(Date and Time, Numeric, String conversion)-String Functions

- **UPPER():** It converts the value of a field to uppercase.
- **Syntax:**
  - SELECT UPPER(column_name) FROM table_name;
- **Example:**
  - SELECT L_Name,Upper(L_Name) AS UCASE_Name FROM Student1;

L_NAME	UCASE_NAME
Ravi	RAVI
Kiran	KIRAN
Sandeep	SANDEEP
Rani	RANI
Manaswi	MANASWI

SQL functions(Date and Time, Numeric, String conversion)-String Functions

- **LOWER():** It converts the value of a field to lowercase.
- **Syntax:**
  - SELECT LOWER(column_name) FROM table_name;
- **Example:**
  - SELECT L_Name,Lower(L_Name) AS LCASE_Name FROM Student1;

L_NAME	LCASE_NAME
Ravi	ravi
Kiran	kiran
Sandeep	sandeep
Rani	rani
Manaswi	manaswi

SQL functions(Date and Time, Numeric, String conversion)-String Functions

- **INITCAP(string):** Converts the first letter of each word to uppercase and the rest to lowercase.
- **Syntax:**
  - SELECT Initcap(column_name) FROM table_name;
- **Example:**
- SELECT L_Name,Initcap(L_Name) AS INITCap_Name FROM Student1;

L_NAME	INITCAP_NAME
Ravi	Ravi
Kiran	Kiran
Sandeep	Sandeep
Rani	Rani
Manaswi	Manaswi

SQL functions(Date and Time, Numeric, String conversion)-String Functions

- **REVERSE():** The REVERSE() function reverses the characters in a string. It's useful in situations where we need to process data backward, such as for password validation or certain pattern matching.
  - **Syntax: SELECT** REVERSE(Column_Name) **AS** Alias_Name **FROM** Table_Name;
  - **Example:** SELECT F_Name,reverse(F_Name) AS Reverse_Name FROM Student1;

F_NAME	REVERSE_NAME
Arun	nurA
Ajay	yajA
Satish	hsitaS
Sandhya	ayhdnaS
Geetha	ahteeG

SQL functions(Date and Time, Numeric, String conversion)-String Functions

- **REPLACE():** In Oracle SQL, the REPLACE function is used to substitute all occurrences of a specified substring within a string with another substring.
  - **Syntax:** REPLACE(original_string, search_string, replacement_string)**Example:**
- **Example:** select F_Name, REPLACE(F_Name,'A','K') as StringRepalce from Student1;

F_NAME	STRINGREPALCE
Arun	Krun
Ajay	Kjay
Satish	Satish
Sandhya	Sandhya
Geetha	Geetha

SQL functions(Date and Time, Numeric, String conversion)-String Functions

- **INSTR():** The INSTR() function returns the position of the first occurrence of a substring within a string. If the substring is not found, it returns 0. It is useful for finding where specific text appears in a larger string.
- **Syntax:** Instr(Column_Name, 'character');
- **Example:** select F_Name, instr(F_Name,'a') as INSTRFormat from Student1;

F_NAME	INSTRFORMAT
Arun	0
Ajay	3
Satish	2
Sandhya	2
Geetha	6

SQL Functions

Date and Time Functions

Function	Example	Result	Description
ADD_MONTHS	ADD_MONTHS(DATE '2016-02-29', 1)	31-MAR-16	Add a number of months (n) to a date and return the same day which is n of months away.
CURRENT_DATE	SELECT CURRENT_DATE FROM dual	06-AUG-2017 19:43:44	Return the current date and time in the session time zone
LAST_DAY	LAST_DAY(DATE '2016-02-01')	29-FEB-16	Gets the last day of the month of a specified date.
MONTHS_BETWEEN	MONTHS_BETWEEN(DATE '2017-07-01', DATE '2017-01-01')	6	Return the number of months between two dates.
NEXT_DAY	NEXT_DAY(DATE '2000-01-01', 'SUNDAY')	02-JAN-00	Get the first weekday that is later than a specified date.
SYSDATE	SYSDATE	01-AUG-17	Return the current system date and time of the operating system where the Oracle Database resides.

SQL Functions

Date and Time Functions

TO_CHAR	TO_CHAR(DATE'2017-01-01', 'DL')	Sunday, January 01, 2017	Convert a DATE or an INTERVAL value to a character string in a specified format.
TO_DATE	TO_DATE('01 Jan 2017', 'DD MON YYYY')	01-JAN-17	Convert a date which is in the character string to a DATE value.
TRUNC	TRUNC(DATE '2017-07-16', 'MM')	01-JUL-17	Return a date truncated to a specific unit of measure.
EXTRACT	EXTRACT(YEAR FROM SYSDATE)	2017	Extract a value of a date time field e.g., YEAR, MONTH, DAY, ... from a date time value.

EXAMPLE

```
1  SELECT EXTRACT(YEAR FROM SYSDATE) FROM dual;
```

EXTRACT(YEARFROMSYSDATE)
2024

SQL functions(Date and Time, Numeric, String conversion)

- **SYSDATE:** Returns the current date and time from the database server.
  - SELECT SYSDATE FROM DUAL;

	SYSDATE
1	8/19/2025, 6:16:10

- **CURRENT_DATE / CURRENT_TIMESTAMP:** Returns the current date or timestamp, often including timezone information.
  - SELECT CURRENT_DATE FROM DUAL;
  - SELECT CURRENT_TIMESTAMP FROM DUAL;

CURRENT_DATE
8/19/2025, 6:17:14

CURRENT_TIMESTAMP
2025-08-19T06:17:49.226484Z

SQL functions(Date and Time, Numeric, String conversion)

- **EXTRACT(part FROM date):** Extracts a specific part (e.g., YEAR, MONTH, DAY, HOUR, MINUTE, SECOND) from a date or timestamp.
  - SELECT EXTRACT(YEAR FROM SYSDATE) AS current_year FROM DUAL;

CURRENT_YEAR
2025

- **ADD_MONTHS(date, n):** Adds or subtracts a specified number of months to a date.
  - SELECT ADD_MONTHS(SYSDATE, 3) AS date_plus_3_months FROM DUAL;

DATE_PLUS_3_MONTHS
11/19/2025, 6:21:46 AM

SQL functions(Date and Time, Numeric, String conversion)

- **LAST_DAY(date):** Returns the last day of the month for a given date.
  - SELECT LAST_DAY(SYSDATE) AS last_day_of_month FROM DUAL;

LAST_DAY_OF_MONTH
8/31/2025, 6:24:35 AM

- **NEXT_DAY(date, 'day_of_week'):** Returns the date of the next specified day of the week following a given date.
  - SELECT NEXT_DAY(SYSDATE, 'MONDAY') AS next_monday FROM DUAL;

NEXT_MONDAY
8/25/2025, 6:25:17

SQL functions(Date and Time, Numeric, String conversion)

- **MONTHS_BETWEEN(date1, date2):** Returns the number of months between two dates.
  - SELECT MONTHS_BETWEEN('01-JAN-2025', '01-JUL-2024') AS months_diff FROM DUAL;

MONTHS_DIFF
6

- **TRUNC(date, 'format_model'):** Truncates a date to a specified unit (e.g., 'YEAR', 'MONTH', 'DD').
  - SELECT TRUNC(SYSDATE, 'MONTH') AS first_day_of_month FROM DUAL;

FIRST_DAY_OF_MONTH
8/1/2025, 12:00:00 AM

SQL functions(Date and Time, Numeric, String conversion)

- **ROUND(date, 'format_model'):** Rounds a date to the nearest specified unit.
  - SELECT ROUND(SYSDATE, 'YEAR') AS rounded_to_year FROM DUAL;

ROUNDED_TO_YEAR
1/1/2026, 12:00:00 AM

- **TO_CHAR(date, 'format_model'):** Converts a date to a character string with a specified format.
  - SELECT TO_CHAR(SYSDATE, 'DD-MON-YYYY HH24:MI:SS') AS formatted_date FROM DUAL;

FORMATTED_DATE
19-AUG-2025 06:32:18

Sub-Queries
or
Nested Queries

Sub-Queries or Nested Queries

1. **A query** is an inquiry into the database using the select command.

2. **A sub query** is a **query within another query** i. e. A query embedded in another query is called a sub query.

3. It is also **known as nested query**.

4. Sub query is used to return data that will be used in the **main query as a condition, to further restrict the data to be retrieved**.

Sub queries are of 2 types :

1. **Independent Nested Query** : Inner query doesn't depend on outer query.

2. **Co-Related Nested Query** : Inner query uses attributes of outer query, hence dependent on outer query.

SQL-Arithmetic & Logical operations

SQL Operators

SQL Arithmetic Operators

We can use various Arithmetic Operators on the data stored in the tables.

Arithmetic Operators are:

```
+          [Addition]
-          [Subtraction]
/          [Division]
*          [Multiplication]
%          [Modulus]
```

SQL Arithmetic Operators

SELECT * **FROM** Employee;

Employee_Id	Emp_Name	Emp_City	Emp_Salary	Emp_bonus
101	Anuj	Ghaziabad	25000	2000
102	Tushar	Lucknow	29000	1000
103	Vivek	Kolkata	35000	2500
104	Shivam	Goa	22000	3000

SQL Arithmetic Operators

Addition (+) :

Write a query adds the Emp_Salary and Emp_Bonus of each employee of the **Employee** table using the addition operator:

```
SELECT Emp_Salary + Emp_Bonus AS Emp_Total_Salary
FROM Employee;
```

Emp_Total_Salary
27000
30000
37500
25000

SQL Arithmetic Operators

Write a query adds 15000 to the salary of each employee in the Emp_Salary column of **the Employee** table:

```
SELECT Emp_Salary + 15000 AS Emp_Updated_Salary
FROM Employee;
```

Emp_Updated_Salary
40000
44000
50000
37000

SQL Arithmetic Operators

SQL Subtraction Operator (-)

The following query subtracts 10000 from the salary of each employee of the **Employee** table:

```
SELECT Emp_Salary - 10000 AS Emp_Updated_Salary FROM Employee;
```

Emp_Updated_Salary
15000
19000
25000
37000

SQL Arithmetic Operators

SQL Multiplication Operator (*)

It is use to perform **multiplication** of data items.

```
SELECT item, amount, amount*4 AS total_amount
FROM Orders;
```

Output

item	amount	total_amount
Keyboard	400	1600
Mouse	300	1200
Monitor	12000	48000
Keyboard	400	1600
Mousepad	250	1000

SQL Arithmetic Operators

SQL Division Operator (/)

The SQL Division operator divides the numerical values of one column by the numerical values of another column.

```
SELECT item, amount, amount/2 AS half_amount
FROM Orders;
```

Output

item	amount	half_amount
Keyboard	400	200
Mouse	300	150
Monitor	12000	6000
Keyboard	400	200
Mousepad	250	125

SQL Arithmetic Operators

SQL Modulus Operator (%)

SQL Modulus Operator provides the remainder when the numerical values of one column are divided by the numerical values of another column.

Student_Id	Student_Name	Student_Maths	Student_English
201	Anuj	30	60
202	Tushar	25	100
203	Vivek	30	90
204	Shivam	40	80

```
SELECT Student_English % Student_Maths AS Remainder FROM Student;
```

Remainder
2
4
3
2

SQL-Logical operations

Operator	Description
ALL	TRUE if all of the subquery values meet the condition
AND	TRUE if all the conditions separated by AND is TRUE
ANY	TRUE if any of the sub query values meet the condition
BETWEEN	TRUE if the operand is within the range of comparisons
IN	TRUE if the operand is equal to one of a list of expressions
LIKE	TRUE if the operand matches a pattern
NOT	Displays a record if the condition(s) is NOT TRUE
OR	TRUE if any of the conditions separated by OR is TRUE
SOME	TRUE if any of the sub query values meet the condition

Consider the CUSTOMERS table

SQL> SELECT * FROM CUSTOMERS;

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

ALL

Operator	Description
ALL	TRUE if all of the sub query values meet the condition

```
Select * from customers
        where age > ALL (select age from customers where
                                        salary > 6500);
```

Output

```
+----+--------+-----+-----------+----------+
| ID | NAME   | AGE | ADDRESS   | SALARY   |
+----+--------+-----+-----------+----------+
|  1 | Ramesh |  32 | Ahmedabad | 2000.00 |
+----+--------+-----+-----------+----------+
```

ANY

ANY	TRUE if any of the sub query values meet the condition

```
select * from customers
        where age > ANY (select age from customers
                                where salary > 6500);
```

Output

```
+----+----------+-----+-----------+---------+
| ID | NAME     | AGE | ADDRESS   | SALARY  |
+----+----------+-----+-----------+---------+
|  1 | Ramesh   |  32 | Ahmedabad | 2000.00 |
|  2 | Khilan   |  25 | Delhi     | 1500.00 |
|  4 | Chaitali |  25 | Mumbai    | 6500.00 |
|  5 | Hardik   |  27 | Bhopal    | 8500.00 |
+----+----------+-----+-----------+---------+
```

BETWEEN

BETWEEN	TRUE if the operand is within the range of comparisons

```
select * from customers
        where age between 25 and 27;
```

```
+----+----------+-----+---------+---------+
| ID | NAME     | AGE | ADDRESS | SALARY  |
+----+----------+-----+---------+---------+
|  2 | Khilan   |  25 | Delhi   | 1500.00 |
|  4 | Chaitali |  25 | Mumbai  | 6500.00 |
|  5 | Hardik   |  27 | Bhopal  | 8500.00 |
+----+----------+-----+---------+---------+
```

IN

IN	TRUE if the operand is equal to one of a list of expressions

```
select * from customers
                where age IN ( 25, 27 );
```

```
+----+----------+-----+---------+----------+
| ID | NAME     | AGE | ADDRESS | SALARY   |
+----+----------+-----+---------+----------+
|  2 | Khilan   |  25 | Delhi   | 1500.00  |
|  4 | Chaitali |  25 | Mumbai  | 6500.00  |
|  5 | Hardik   |  27 | Bhopal  | 8500.00  |
+----+----------+-----+---------+----------+
```

SOME

1. SOME operator evaluates the condition between the outer and inner tables.

2. It Evaluates to true if the final result returns **any one** row. If not, then it evaluates to false.

```
SELECT column_name(s)
FROM table_name
WHERE expression comparison_operator SOME (subquery)
```

EXAMPLE

Instructor Table:

Name	Department	Salary
Chandra	Computational Biology	1
Visweswaran	Electronics	1.5
Abraham	Computer Science	1.3
John	Electronics	1.2
Samantha	Computer Science	2
Jyoti	Electronics	1.2
Debarka	Computer Science	2
Ganesh	Computational Biology	0.9

EXAMPLE

Sample Queries and Outputs:

```
select name
from instructor
where Salary > some(select Salary
from instructor
where dept='Computer Science');
```

Output:

Visweswaran
Samantha
Debarka

SQL
EXISTS & NOTEXISTS

SQL EXISTS Operator

1. The EXISTS operator is used to test for the existence of any record in a sub query.
2. The EXISTS operator returns TRUE if the sub query returns one or more records.

SQL EXISTS syntax

```
SELECT column_name
FROM Table_Name
WHERE EXISTS (SELECT column_name FROM Table_Name WHERE condition);
```

SQL EXISTS Operator

Examples: **Consider the following two relation "Customers" and "Orders".**

Customers

customer_id	lname	fname	website
401	Singh	Dolly	abc.com
402	Chauhan	Anuj	def.com
403	Kumar	Niteesh	ghi.com
404	Gupta	Shubham	jkl.com
405	Walecha	Divya	abc.com
406	Jain	Sandeep	jkl.com
407	Mehta	Rajiv	abc.com
408	Mehra	Anand	abc.com

Orders

order_id	c_id	order_date
1	407	2017-03-03
2	405	2017-03-05
3	408	2017-01-18
4	404	2017-02-05

SQL EXISTS Operator

Using EXISTS condition with SELECT statement To fetch the first and last name of the customers who placed at least one order.

```
SELECT fname, lname
FROM Customers
WHERE EXISTS (SELECT *
              FROM Orders
              WHERE Customers.customer_id = Orders.c_id);
```

Output:

fname	lname
Shubham	Gupta
Divya	Walecha
Rajiv	Mehta
Anand	Mehra

SQL NOT EXISTS Operator

Using NOT with EXISTS Fetch last and first name of the customers who has not placed any order.

```
SELECT lname, fname
FROM Customers
WHERE NOT EXISTS (SELECT *
                  FROM Orders
                  WHERE Customers.customer_id = Orders.c_id);
```

Output:

lname	fname
Singh	Dolly
Chauhan	Anuj
Kumar	Niteesh
Jain	Sandeep

SQL EXISTS Operator

Using EXISTS condition with DELETE statement Delete the record of all the customer from Order Table whose last name is 'Mehra'.

```
DELETE
FROM Orders
WHERE EXISTS (SELECT *
              FROM customers
              WHERE Customers.customer_id = Orders.c_id
              AND Customers.lname = 'Mehra');
```

Output:

order_id	c_id	order_date
1	407	2017-03-03
2	405	2017-03-05
4	404	2017-02-05

Set Operators in SQL

SQL UNION Operator

The UNION operator is used to **combine the result-set** of two or more SELECT statements.

1. Each SELECT statement within UNION must have the **same number of columns**

2. The columns must also **have similar data types**

3. The columns in each SELECT statement must also be in **the same order.**

4. The UNION operator selects only distinct values **but ALL keyword is used with UNION to allow duplicate values.**

Set Operators in SQL

Operator	Returns
UNION	All distinct rows selected by either query
UNION ALL	All rows selected by either query, including all duplicates
INTERSECT	All distinct rows selected by both queries
MINUS	All distinct rows selected by the first query but not the second

Syntax

UNION Syntax

SELECT *column_name(s)* FROM *table1*
UNION
SELECT *column_name(s)* FROM *table2*;

UNION ALL Syntax

The UNION operator selects only distinct values by default. To allow duplicate values, use UNION ALL:

SELECT *column_name(s)* FROM *table1*
UNION ALL
SELECT *column_name(s)* FROM *table2*;

Table1 : 'Professors' Table2 : 'Students'

Prof_ID	Prof_Name	City	Country
1	Thomas	Hamilton	New Zealand
2	Albert	Perth	Australia
3	Bush	Sydney	Australia
4	Clinton	Sydney	Australia
5	Fleming	Hamilton	New Zealand

Stud_ID	Stud_Name	City	Country
1	Mark	London	England
2	Alex	Paris	France
3	Bob	Sydney	Australia
4	Jaya	Delhi	India
5	Surendra	Baroda	India

EXAMPLE::

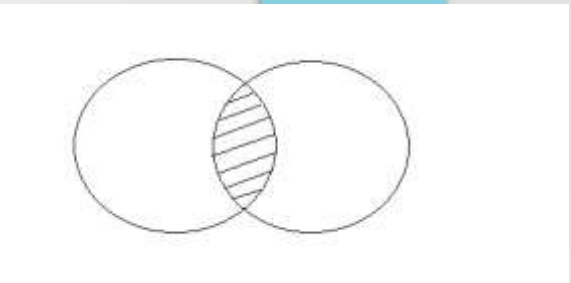
1.Write a query to select different cities from table1 and table2.

```
SELECT City FROM  Professors
UNION
SELECT City FROM Students;
```

2. Write a query to select cities by using ALL keyword.

```
SELECT City FROM  Professors
UNION ALL
SELECT City FROM Students;
```

SQL INTERSECT clause/operator



The SQL **INTERSECT** clause/operator is used to combine two SELECT statements, but returns rows only from the first SELECT statement that are identical to a row in the second SELECT statement.

1. This means INTERSECT returns only common rows returned by the two SELECT statements.

2. Just as with the UNION operator, the same rules apply when using the INTERSECT operator.

3. MySQL does not support the INTERSECT operator.

Syntax

The basic syntax of **INTERSECT** is as follows.

```
SELECT column1 [, column2 ]
FROM table1 [, table2 ]
[WHERE condition]

INTERSECT

SELECT column1 [, column2 ]
FROM table1 [, table2 ]
[WHERE condition]
```

Example of Intersect

The **First** table,

ID	NAME
1	abhi
2	adam

The **Second** table,

ID	NAME
2	adam
3	Chester

```
select * from First
INTERSECT
select * from second
```

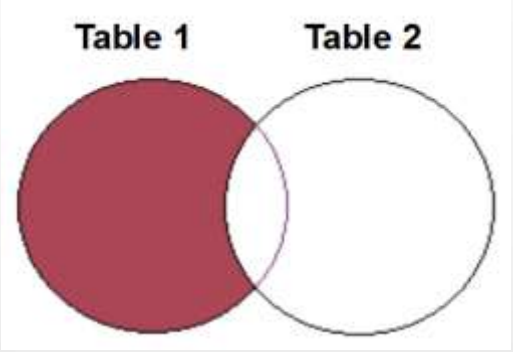
ID	NAME
2	adam

SQL MINUS Operator

1. The Minus Operator in SQL is used with two SELECT statements.

2. The MINUS operator is used to subtract the result set obtained by first SELECT query from the result set obtained by second SELECT query.

3. In simple words, we can say that **MINUS operator will return only those rows which are unique in only first SELECT query** and not those rows which are common to both first and second SELECT queries.



Basic Syntax:

```
SELECT column1 , column2 , ... columnN
FROM table_name
WHERE condition

MINUS

SELECT column1 , column2 , ... columnN
FROM table_name
WHERE condition; columnN
```

Important Points:

- The WHERE clause is optional in the above query.
- The number of columns in both SELECT statements must be same.
- The data type of corresponding columns of both SELECT statement must be same.

Example of Minus

The **First** table,

ID	NAME
1	abhi
2	adam

The **Second** table,

ID	NAME
2	adam
3	Chester

```
select * from First
MINUS
select * from second
```

ID	NAME
1	abhi

SQL AS Keyword

(Defining a Column Alias)

SQL AS Keyword

1. The AS command is used to rename a column or table with an **alias.**
2. SQL 'AS' is used to assign a new name temporarily to a table column or even a table.
3. Aliases are often used to make column names **more readable**.
4. An alias only exists for the **duration of that query**.
5. An alias is created with the **AS keyword**.

SQL AS Keyword

Syntax

When alias is used on column:

SELECT *column_name* AS *alias_name*
FROM *table_name;*

Example

SELECT CustomerID AS ID, CustomerName AS Customer
FROM Customers;

SQL Joins

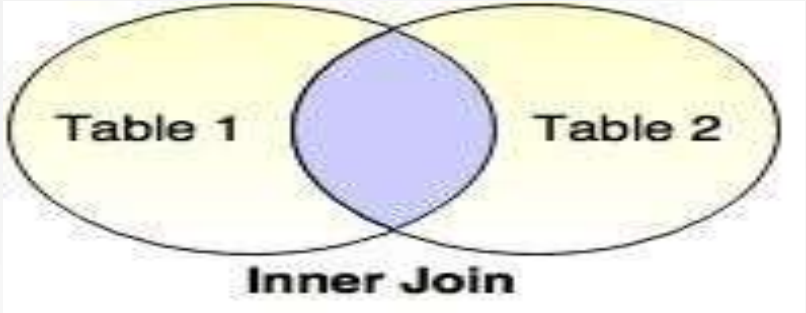
SQL Joins

A **JOIN** clause is used to combine rows from two or more tables, based on a related column between them.

Types of Joins in DBMS

1. **Inner Join**
2. **Natural Join**
3. **Equi Join**
4. **Outer Joins**
   - Left Outer Join
   - Right Outer Join
   - Full Outer Join
5. **Cross Join (Cartesian Join)**

SQL INNER JOIN Statement



1. SQL INNER JOIN statement selects all possible rows from both tables, if there is a match between the columns in both tables.

2. The data should **not be displayed, if there is no match** between columns in both columns.

Syntax:

```
SELECT column_name

FROM table1 INNER JOIN table2

ON table1.column_name = table2.column_name;
```

Example : Query using INNER JOIN Statement.

Consider the following table titled 'Employee'.

Emp_ID	Emp_Name	Emp_Phone
1	Marten	9022222222
2	Paul	9033333333
3	Albert	9044444444
4	Steve	9055555555

Consider the following table titled 'Salary'.

Emp_ID	Transaction_ID	Amount
1	101	10000
2	102	20000
3	103	25000
4	104	30000
5	105	30000
6	106	50000

Example : Query using INNER JOIN Statement.

The column 'Emp_ID' is common to both tables. So the query should be written as:

```
SELECT  Employee.Emp_Name,  Salary.Transaction_ID,
Salary.Amount

FROM Employee
INNER JOIN Salary

ON Salary.Emp_ID =  Employee.EMP_ID;
```

Emp_Name	Transaction_ID	Amount
Marten	101	10000
Paul	102	20000
Albert	103	25000
Steve	105	30000

Natural JOIN

1. Joins two tables based on all columns with the **same name and datatype**.
2. **Removes duplicate** columns from the result.

It is called "natural" because the join condition is **implicitly chosen** (no need to write ON clause).

The syntax for Natural Join is,

```
SELECT * FROM tablename1
NATURAL JOIN tablename2;
```

Example of Natural JOIN

Here is the **class** table,

ID	NAME
1	abhi
2	adam
3	alex
4	anu

and the **class_info** table,

ID	Address
1	DELHI
2	MUMBAI
3	CHENNAI

Example of Natural JOIN

Natural join query will be,

SELECT * from class NATURAL JOIN class_info;

The resultset table will look like,

ID	NAME	Address
1	abhi	DELHI
2	adam	MUMBAI
3	alex	CHENNAI

In the above example, both the tables being joined have ID column(same name and same datatype)

Equi Join

- An **Equi Join** is a type of **Inner Join** that uses an **equality condition (=)** to match rows from two tables.

- Unlike **Natural Join**, it does **not automatically remove duplicate join columns**.

Example

Student

student_id	name	dept_id
1	Ravi	10
2	Anu	20
3	Kiran	10

Department

dept_id	dept_name
10	CSE
20	ECE
30	Mechanical

Equi Join

Query (Equi Join)

```
SELECT Student.student_id, Student.name, Student.dept_id, Department.dept_name
FROM Student, Department
WHERE Student.dept_id = Department.dept_id;
```

Result

student_id	name	dept_id	dept_name
1	Ravi	10	CSE
2	Anu	20	ECE
3	Kiran	10	CSE

Difference Between Inner Join and Equi Join

Example

Student(student_id, name, dept_id)
Department(dept_id, dept_name)

Inner Join

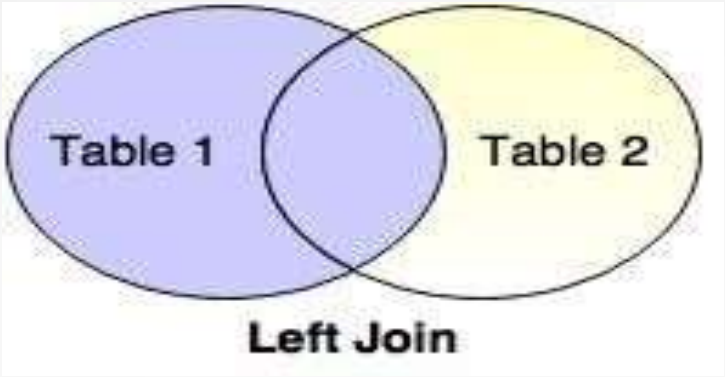
```sql
SELECT S.name, D.dept_name
FROM Student S
INNER JOIN Department D
ON S.dept_id = D.dept_id;
```

Equi Join

```sql
SELECT S.name, D.dept_name
FROM Student S, Department D
WHERE S.dept_id = D.dept_id;
```

Both give the **same result**, but Equi Join is **restricted to equality** while Inner Join is more **general**.

LEFT JOIN



1. SQL LEFT JOIN or LEFT OUTER JOIN keyword returns all rows from the left table (table1) with the matching rows in the right table (table2).

2. If there is no match found, the result displayed as NULL if no match found on right column.

Syntax:

```
SELECT column_name

FROM table1

LEFT JOIN table2

ON table1.column_name = table2.column_name;
```

Example : Query using LEFT JOIN Statement.

Consider the following table titled 'Employee'.

Emp_ID	Emp_Name	Emp_Phone
1	Marten	9022222222
2	Paul	9033333333
3	Albert	9044444444
4	Steve	9055555555

Consider the following table titled 'Salary'.

Emp_ID	Transaction_ID	Amount
1	101	10000
2	102	20000
3	103	25000
4	104	30000
5	105	30000
6	106	50000

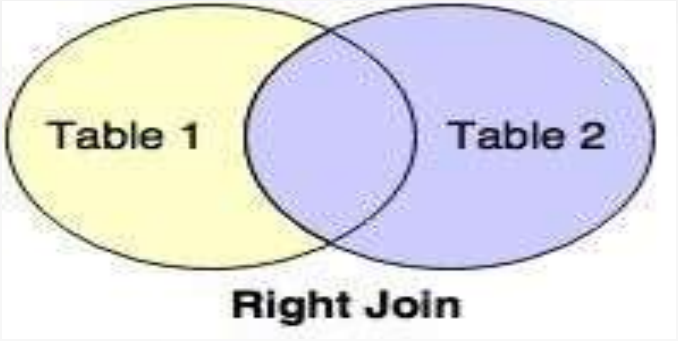
Example : Query using LEFT JOIN Statement.

To perform LEFT JOIN on table (Salary), query should be written as:

```
SELECT Employee.Emp_Name, Employee.Emp_Phone, Salary.Amount FROM
Employee
LEFT JOIN Salary
ON Salary.Emp_ID = Employee.Emp_ID;
```

Emp_Name	Emp_Phone	Amount
Marten	9022222222	10000
Paul	9033333333	20000
Albert	9044444444	25000
Steve	9055555555	30000

RIGHT JOIN



1. The SQL RIGHT JOIN or RIGHT OUTER JOIN keyword returns all rows from the right table (table2), with the matching rows in the left table (table1).

2. The result is null in the left table, if there is no match.

Syntax:

```
SELECT column_name

FROM table1

RIGHT JOIN table2

ON table1.column_name = table2.column_name;
```

Example : Query using RIGHT JOIN Statement.

Consider the following table entitled 'Employee'.

Emp_ID	Emp_Name	Emp_Phone
1	Marten	9022222222
2	Paul	9033333333
3	Albert	9044444444
4	Steve	9055555555
5	Smith	9066666666
6	Jhon	9011111111

Consider the following table entitled as 'Salary'.

Emp_ID	Transaction_ID	Amount
1	101	10000
2	102	20000
3	103	25000
4	104	30000
5	105	30000

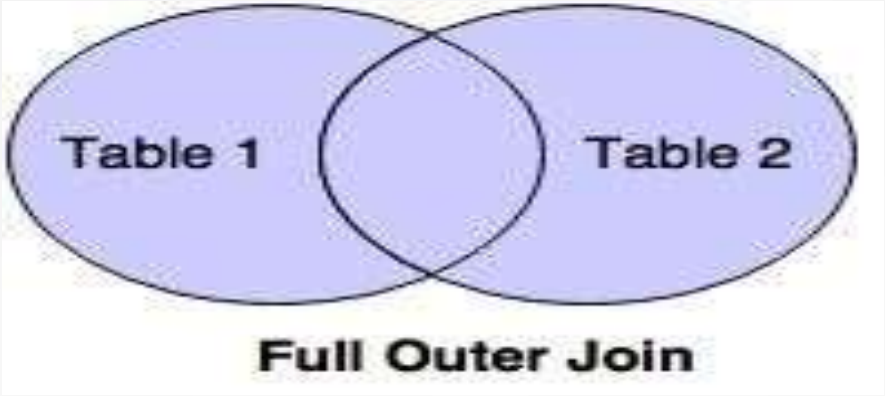
Example : Query using RIGHT JOIN Statement.

To perform RIGHT JOIN ON table (Employee), query should be written as:

```
SELECT    Employee.Emp_Name,  Salary.Amount
FROM  Salary
RIGHT JOIN Employee
ON  Salary.Emp_ID =  Employee.Emp_ID;
```

Emp_Name	Amount
Martin	10000
Paul	20000
Albert	25000
Steve	30000
Smith	30000
Jhon	

SQL FULL JOIN



1. The FULL JOIN or FULL OUTER JOIN statement returns all rows from the left table (table1) and the right table (table2).

Syntax:

```
SELECT column_name

FROM table1

FULL JOIN table2

ON table1.column_name = table2.column_name;
```

Example : Query using FULL JOIN Statement.

Consider the following table entitled 'Employee'.

Emp_ID	Emp_Name	Emp_Phone
1	Marten	9022222222
2	Paul	9033333333
3	Albert	9044444444
4	Steve	9055555555
5	Smith	9066666666
6	Jhon	9011111111

Consider the following table entitled as 'Salary'.

Emp_ID	Transaction_ID	Amount
1	101	10000
2	102	20000
3	103	25000
4	104	30000

Example : Query using FULL JOIN Statement.

To perform FULL JOIN, query should be written as:

```
SELECT Employee.Emp_ID,
Employee.Emp_Name,
Salary.Transaction_ID, Salary.Amount
FROM Employee
FULL JOIN Salary
ON Salary.Emp_ID = Employee.Emp_ID;
```

Emp_ID	Emp_Name	Transaction_ID	Amount
1	Marten	101	10000
2	Paul	102	20000
3	Albert	103	25000
4	Steve	104	30000
5	Smith		
6	Jhon		

CROSS JOIN

1. The SQL CROSS JOIN returns the cartesian product of rows from joined table.

2. A cartesian join is a join of all rows of table1 to all rows of table2.

Syntax:

```
SELECT * FROM table1 CROSS JOIN table2;
```

Example : Query using CROSS JOIN Statement.

Table1 : Students

Stud_ID	Stud_Name	Subject	Dept_ID
1	Ravi	C	01
2	Sapna	CPP	02
3	Prajakta	JAVA	03

Table2 : Department

Dept_ID	Dept_Name
01	Comp_Science
02	IT
03	MCA

Example : Query using CROSS JOIN Statement.

```
SELECT Std_Id, Stud_Name, Subject,
Dept_Name

FROM students CROSS JOIN Department;
```

Stud_ID	Stud_Name	Subject	Dept_Name
1	Ravi	C	Comp_Science
2	Sapna	CPP	Comp_Science
3	Prajakta	JAVA	Comp_Science
1	Ravi	C	IT
2	Sapna	CPP	IT
3	Prajakta	JAVA	IT
1	Ravi	C	MCA
2	Sapna	CPP	MCA
3	Prajakta	JAVA	MCA

JOINS EXAMPLES ::We will use two sample tables:

Table: Students

StudentID	Name	DeptID
1	Ravi	10
2	Sita	20
3	Arjun	30
4	Meena	NULL

Table: Departments

DeptID	DeptName
10	CSE
20	ECE
40	Mechanical

INNER JOIN EXAMPLES

Q1. Show all students with their department names.

```sql
SELECT Students.Name, Departments.DeptName
FROM Students
INNER JOIN Departments
ON Students.DeptID = Departments.DeptID;
```

✅ **Output:**

Name	DeptName
Ravi	CSE
Sita	ECE

LEFT JOIN EXAMPLES

Q2. List all students and their department names (include students with no department).

```sql
SELECT Students.Name, Departments.DeptName
FROM Students
LEFT JOIN Departments
ON Students.DeptID = Departments.DeptID;
```

✅ **Output:**

Name	DeptName
Ravi	CSE
Sita	ECE
Arjun	NULL
Meena	NULL

RIGHT JOIN EXAMPLES

Q3. Show all departments and the students in them (even if no student belongs).

```
SELECT Students.Name, Departments.DeptName
FROM Students
RIGHT JOIN Departments
ON Students.DeptID = Departments.DeptID;
```

✅ **Output:**

Name	DeptName
Ravi	CSE
Sita	ECE
NULL	Mechanical

FULL JOIN EXAMPLES

Q4. Display all students and departments (including unmatched ones).

```sql
SELECT Students.Name, Departments.DeptName
FROM Students
FULL OUTER JOIN Departments
ON Students.DeptID = Departments.DeptID;
```

✅ **Output:**

Name	DeptName
Ravi	CSE
Sita	ECE
Arjun	NULL
Meena	NULL
NULL	Mechanical

View
(updatable and non-updatable)

VIEWS

Definition of View in DBMS

1. A **view** is a **virtual table** in a database that is created by writing a SQL query.

2. It does **not store data physically**; instead, it displays data stored in one or more base tables.

3. Views are created by the database user (developer, DBA, or authorized person) who has the CREATE VIEW privilege.

Syntax of View

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Example

Step 1: Create Base Table

```sql
CREATE TABLE Employees (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    Salary DECIMAL(10,2),
    DeptID INT
);
```

```sql
INSERT INTO Employees VALUES
(1, 'Ramesh', 50000, 101),
(2, 'Sita', 60000, 102),
(3, 'Arun', 55000, 101),
(4, 'Meena', 70000, 102);
```

Example

Step 2: Create a View

```
CREATE VIEW HighSalaryEmployees AS
SELECT EmpID, EmpName, Salary
FROM Employees
WHERE Salary > 55000;
```

Step 3: Use the View

```
SELECT * FROM HighSalaryEmployees;
```

Output:

EmpID	EmpName	Salary
2	Sita	60000
4	Meena	70000

View(updatable and non-updatable)

Updatable View

Definition

An **updatable view** is a view where you can perform **INSERT, UPDATE, and DELETE** operations.

Changes made in the view are reflected in the base table.

Conditions for Updatable View:

1. Based on **one table only**.
2. Contains the **primary key** of the table.
3. Does not use DISTINCT, GROUP BY, HAVING, UNION, JOIN, or aggregate functions.

Example (Updatable View)

Base Table:

```sql
CREATE TABLE Employees (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    Salary DECIMAL(10,2),
    DeptID INT
);
```

```sql
INSERT INTO Employees VALUES
(1, 'Ramesh', 50000, 101),
(2, 'Sita', 60000, 102),
(3, 'Arun', 55000, 101);
```

Create Updatable View:

```sql
CREATE VIEW EmpView AS
SELECT EmpID, EmpName, Salary
FROM Employees;
```

Example (Updatable View)

Update using View:

```
UPDATE EmpView
SET Salary = 65000
WHERE EmpID = 2;
```

This will update **Sita's salary** in both EmpView and Employees.

Delete using View:

```
DELETE FROM EmpView
WHERE EmpID = 1;
```

This will remove **Ramesh** from both EmpView and the base table.

Non-Updatable View

A **non-updatable view** is a view where you **cannot perform INSERT, UPDATE, or DELETE** operations.

Reasons for Non-Updatable View:

- Uses **aggregates** (SUM, AVG, MAX, MIN).
- Uses **DISTINCT, GROUP BY, HAVING**.
- Based on **multiple tables (JOINs)**.
- Does not include a **primary key**.

Example (Non-Updatable View)

Create Non-Updatable View:

```
CREATE VIEW DeptSalary AS
SELECT DeptID, AVG(Salary) AS AvgSalary
FROM Employees
GROUP BY DeptID;
```

Result of View:

DeptID	AvgSalary
101	52500.00
102	60000.00

Try Update:

```
UPDATE DeptSalary
SET AvgSalary = 70000
WHERE DeptID = 101;
```

This will give an **error** because AvgSalary is **calculated** and does not exist in the base table.

View(updatable and non-updatable)

- Examples

View(updatable and non-updatable)

- CREATE TABLE CUSTOMERS(

```
 ID   INT                NOT NULL,
NAME VARCHAR (20)     NOT NULL,
AGE  INT                NOT NULL,
ADDRESS  CHAR (25) ,
SALARY   DECIMAL (18, 2),
PRIMARY KEY (ID)
);
```

View(updatable and non-updatable)

- INSERT INTO CUSTOMERS VALUES

(1, 'Ramesh', 32, 'Ahmedabad', 2000.00),

(2, 'Khilan', 25, 'Delhi', 1500.00),

(3, 'Kaushik', 23, 'Kota', 2000.00),

(4, 'Chaitali', 25, 'Mumbai', 6500.00),

(5, 'Hardik', 27, 'Bhopal', 8500.00),

(6, 'Komal', 22, 'Hyderabad', 4500.00),

(7, 'Muffy', 24, 'Indore', 10000.00);

View(updatable and non-updatable)

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000
2	Khilan	25	Delhi	1500
3	Kaushik	23	Kota	2000
4	Chaitali	25	Mumbai	6500
5	Hardik	27	Bhopal	8500
6	Komal	22	Hyderabad	4500
7	Muffy	24	Indore	10000

View(updatable and non-updatable)

- create view CustView as

  select Name, Age, Salary from Customers;

- */create view CustView as select * from Customers; */

NAME	AGE	SALARY
Ramesh	32	2000
Khilan	25	1500
Kaushik	23	2000
Chaitali	25	6500
Hardik	27	8500
Komal	22	4500
Muffy	24	10000

View(updatable and non-updatable)

- Select name from CustView;

NAME
Ramesh
Khilan
Kaushik
Chaitali
Hardik
Komal
Muffy

View(updatable and non-updatable)

- create view Custview1 as

  select * from Customers where age>24;

- Select * from Custview1;

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000
2	Khilan	25	Delhi	1500
4	Chaitali	25	Mumbai	6500
5	Hardik	27	Bhopal	8500

View(updatable and non-updatable)

- UPDATE Custview1

  SET Age=40 where Name='Ramesh';

- select * from CustView1;

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	40	Ahmedabad	2000
2	Khilan	25	Delhi	1500
4	Chaitali	25	Mumbai	6500
5	Hardik	27	Bhopal	8500

View(updatable and non-updatable)

- select * from Customers;

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	40	Ahmedabad	2000
2	Khilan	25	Delhi	1500
3	Kaushik	23	Kota	2000
4	Chaitali	25	Mumbai	6500
5	Hardik	27	Bhopal	8500
6	Komal	22	Hyderabad	4500
7	Muffy	24	Indore	10000

View(updatable and non-updatable)

- insert into Custview1 values (8,'Satish',29,'Delhi',8000);

- select * from Custview1;

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	40	Ahmedabad	2000
2	Khilan	25	Delhi	1500
4	Chaitali	25	Mumbai	6500
5	Hardik	27	Bhopal	8500
8	Satish	29	Delhi	8000

View(updatable and non-updatable)

- select * from Customers;

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	40	Ahmedabad	2000
2	Khilan	25	Delhi	1500
3	Kaushik	23	Kota	2000
4	Chaitali	23 25	Mumbai	6500
5	Hardik	27	Bhopal	8500
6	Komal	22	Hyderabad	4500
7	Muffy	24	Indore	10000
8	Satish	29	Delhi	8000

View(updatable and non-updatable)

- delete from Custview1 where name='Satish';

- Select * from Custview1;

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	40	Ahmedabad	2000
2	Khilan	25	Delhi	1500
4	Chaitali	25	Mumbai	6500
5	Hardik	27	Bhopal	8500

View(updatable and non-updatable)

- select * from Customers;

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	40	Ahmedabad	2000
2	Khilan	25	Delhi	1500
3	Kaushik	23	Kota	2000
4	Chaitali	25	Mumbai	6500
5	Hardik	27	Bhopal	8500
6	Komal	22	Hyderabad	4500
7	Muffy	24	Indore	10000

View(updatable and non-updatable)

- delete from Customers where name='Muffy';

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	40	Ahmedabad	2000
2	Khilan	25	Delhi	1500
3	Kaushik	23	Kota	2000
4	Chaitali	25	Mumbai	6500
5	Hardik	27	Bhopal	8500
6	Komal	22	Hyderabad	4500

View(updatable and non-updatable)

- select * from Custview1;

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	40	Ahmedabad	2000
2	Khilan	25	Delhi	1500
4	Chaitali	25	Mumbai	6500
5	Hardik	27	Bhopal	8500

View(updatable and non-updatable)

- delete from Customers where name='Hardik';
- select * from Customers;

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	40	Ahmedabad	2000
2	Khilan	25	Delhi	1500
3	Kaushik	23	Kota	2000
4	Chaitali	25	Mumbai	6500
6	Komal	22	Hyderabad	4500

View(updatable and non-updatable)

- select * from Custview1;

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	40	Ahmedabad	2000
2	Khilan	25	Delhi	1500
4	Chaitali	25	Mumbai	6500